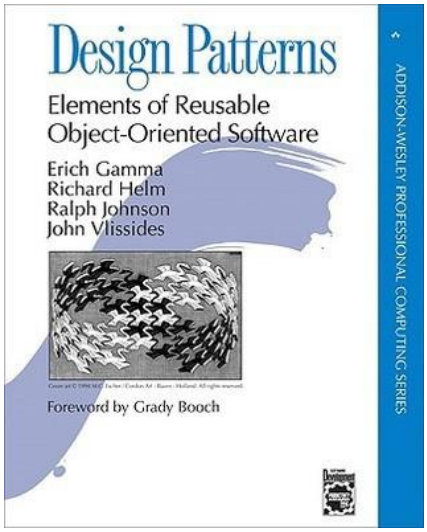
Práctica 7

Patrones de diseño

1. Patrones de diseño GOF

Los patrones de diseño de la “Pandilla de los cuatro” o GOF proceden del libro “Design Patterns”, escrito por cuatro autores: Gamma, Helm, Johnson y Vlissides.

Este libro básico y muy popular presenta 23 patrones que son útiles durante el diseño de objetos. Es importante aprender estos patrones para obtener un vocabulario de diseño de software compartido más rico en término de nombre de patrones, lo que ayuda a comunicar mejores ideas de diseño.



Los patrones de diseño en general describen problemas de diseño recurrentes, que surgen en un contexto de diseño específico, y proponen soluciones. Los 23 patrones GOF se enfocan en describir la estructura, las relaciones y la interacción entre los componentes para resolver un problema de diseño dentro de un contexto definido.

En el libro se clasifican los patrones en tres categorías: patrones creacionales, estructurales y de comportamiento.

2. Patrones creacionales

Los patrones creacionales son los que tienen que ver con los mecanismos de creación de objetos. Entre los patrones creacionales tenemos: Factory Method, Abstract Factory, Builder, Prototype y Singleton.

Estos patrones se utilizan para quitarle al cliente el problema de tener que construir un objeto, y ayudan a que la clase cliente sea más cohesiva, más pequeña y menos acoplada.

Todos los patrones creacionales, excepto el Singleton, presentan una solución donde se pone una capa intermedia de clases, para que el cliente no conozca nunca los productos concretos, ellos devuelven un producto polimórfico.

3. Patrones estructurales

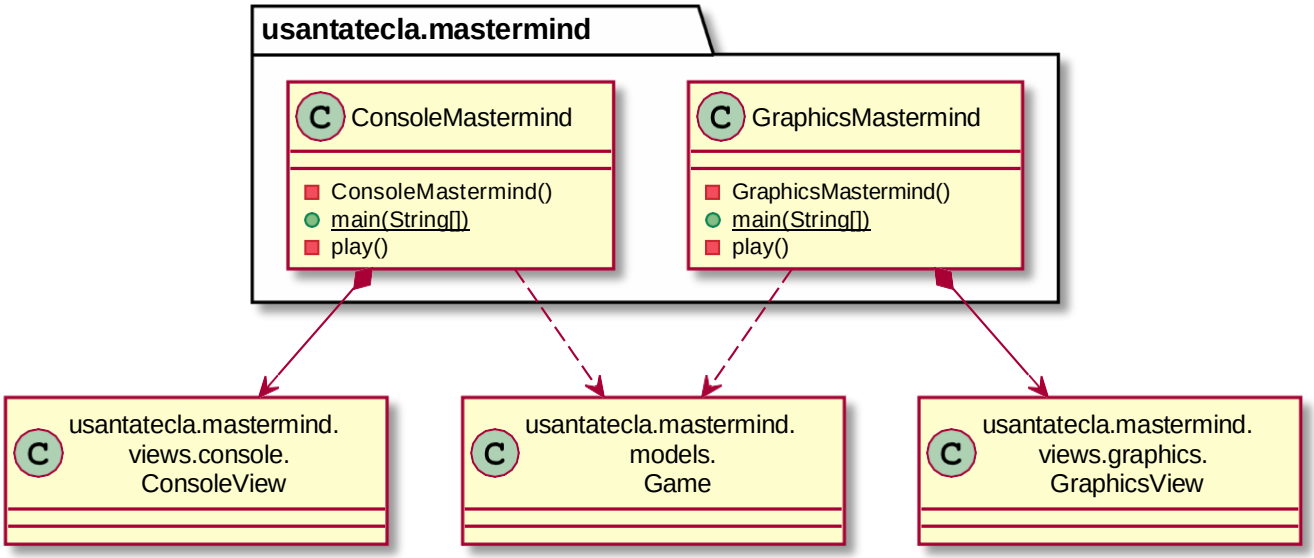
Los patrones estructurales facilitan el diseño mediante la identificación de las relaciones entre las entidades, y tienen que ver con la estructura de los objetos y clases. Entre los patrones estructurales tenemos: Adapter, Bridge, Composite, Decorator, Facade, Flyweight y Proxy.

4. Patrones de Comportamiento

Los patrones de comportamiento identifican patrones comunes de interacción entre los objetos. Estos patrones, al distribuir responsabilidades, aumentan la flexibilidad en la realización de las interacciones. Entre los patrones de comportamiento tenemos: Interpreter, Template Method, Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy y Visitor.

5. Ejercicios

a) La implementación de una versión de Mastermind permite inicial el juego para interactúar vía consola o vía interfaz gráfica. Las clases creadas para esto se pueden ver en el siguiente diagrama UML:



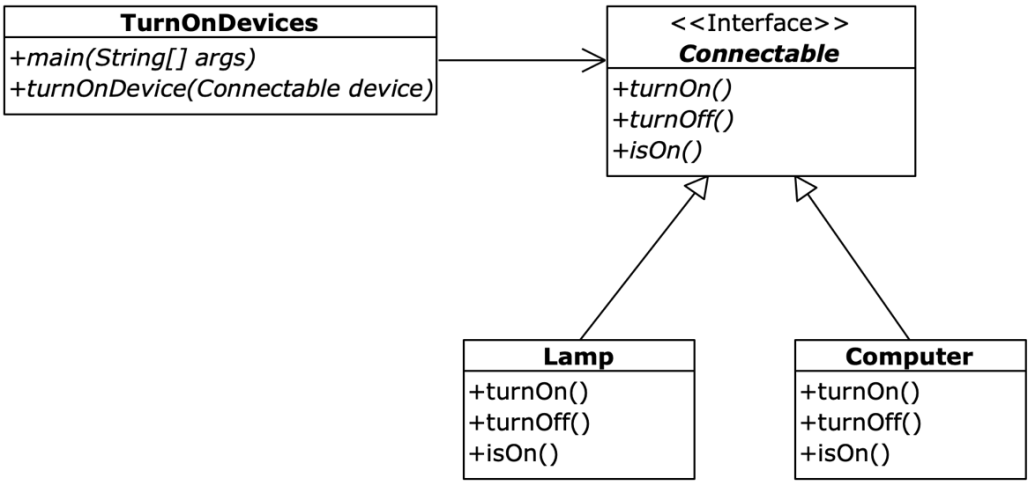
Una implementación de las clases Console Mastermind y Graphics Mastermind es la siguiente:

```
classConsoleMastermind{

    privateBoardboard;
    privateConsoleViewconsoleView;

    privateConsoleMastermind(){
        this.board=newBoard();
        this.consoleView=newConsoleView(this.board);
    }

    privatevoidplay(){ do
        {
            this.consoleView.start();
            this.consoleView.play();
        }while(this.consoleView.resume());
    }

    publicstaticvoidmain(String[]args){ new
        ConsoleMastermind().play();
    }

}

classGraphicsMastermind{

    privateBoardboard;
    privateGraphicsViewgraphicsView;

    privateGraphicsMastermind(){
        this.board = new Board();
        this.graphicsView=newGraphicsView(this.board);
    }

    privatevoidplay(){ do
        {
            this.graphicsView.start();
            this.graphicsView.play();
        }while(this.graphicsView.resume());
    }

    publicstaticvoidmain(String[]args){ new
        GraphicsMastermind().play();
    }
}
```

En esta implementación podemos observar que se repite estructura de código (métodos play y main, atributos). En busca de un mejor diseño del código y evitar código repetido, el ingeniero de software del proyecto le solicita que se utilice el patrón de diseño factoría y cree una solución mejor.

b) En un proyecto de software se necesita crear una clase **Config** que permita configurar parámetros generales de la aplicación. El configurador deberá almacenará los valores necesarios para conectarse a una base de datos: nombre, url, usuario y contraseña. Dado que los valores de configuración se mantienen y son compartidos por toda la aplicación, esta clase debe ser instanciada una sola vez. La clase también debe tener métodos para obtener o modificar los valores. Cree esta clase y utilice el patrón Singleton para lograr el objetivo.

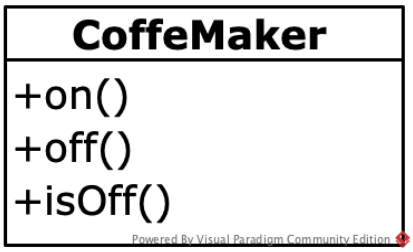
c) Un desarrollador que está creando una aplicación para una casa inteligente tiene creadas las siguientes clases e interfaz:



Donde, la implementación de la clase *TurnOnDevices* es la siguiente:

```
publicclassTurnOnDevices{

  publicstaticvoidmain(String[]args){

     turnOnDevice(new Lamp());
    turnOnDevice(newComputer());

  }

  privatestaticvoidturnOnDevice(Connectabledevice){ device.turnOn();
    System.out.println(device.isOn());
  }

}
```

El cliente le ha solicitado que agregue un nuevo dispositivo, una cafetera, pero la clase que la prende y apaga no cumple con la interfaz *Connectable*, y como ese código pertenece a una librearía de un tercero no lo puede modificar. La clase en cuestión tiene la siguiente forma:



El desarrollador le pide ayuda para solucionar este problema. Utilice el patrón Adaptador para poder usar el método turnOnDevice de la clase *TurnOnDevices* en el código ya existente. Cree el diagrama UML nuevo y el código de la clase *CoffeMaker* resultante en Java. Al terminar, agregue en la clase *TurnOnDevices* la línea nueva para incluir el encendido del dispositivo *CoffeMaker*.